

Stato Project Status Report

leadgeneration.io · Phase 1

Prepared by: Yash Chavan

Date: 23 April 2026

Status: Phase 1 — Pre-Launch

Stato — Project Status Report

Date: 23 April 2026
Prepared by: Yash Chavan
Client: Sam (leadgeneration.io / WinTrading)
Phase: 1 (leadgeneration.io go-live)

1. Executive Summary

Stato is now functionally complete for Phase 1 go-live. All six previously-mock service areas (SOPs, Tasks, Staff/HR, Workflows, Traffic Sources, Client Portal) are now backed by real PostgreSQL tables. Eight third-party integrations have been built; five are live and verified, three are gated on deliverables from Sam.

The codebase is at **383/383 backend tests passing** with **zero TypeScript errors**. Two database migrations (0004, 0005) and one full deploy runbook have been written. All work is committed and pushed to `Development` on both repositories.

The remaining path to go-live is primarily operational: provision Railway prod environment, set secrets, run preflight, and exercise the SignNow paid-plan upgrade before the trial expires on **29 April 2026** (six days away).

1.1 Headline Numbers

Metric	Value
Backend tests passing	383 / 383 (100%)
TypeScript errors	0
Database tables	26 (8 added this sprint)
Migrations applied to dev DB	0000 → 0005
Integrations built	8
Integrations live	5
Integrations awaiting Sam	3
Commits this sprint	11 (9 backend, 2 frontend)

1.2 Top Three Risks

1. **SignNow trial expires 29 April 2026.** Without a paid-plan upgrade, document signing breaks the day after expiry. Owner: Sam.
2. **LeadByte still in mock mode.** All campaign / lead / supplier features fall back to canned data until Sam delivers `LEADBYTE_API_KEY`. Owner: Sam.
3. **JWT secrets not yet generated for production.** Backend will refuse to start in `NODE_ENV=production` if secrets equal dev defaults — must run `openssl rand -base64 32` twice and load into Railway before first deploy. Owner: Yash.

2. Integration Status

Each integration has a real client implementation in code; behaviour at runtime depends on whether the required environment variables are set. Five integrations are gated by Sam delivering credentials or completing a sign-up step.

2.1 Live Integrations

Integration	Code	Live in <code>.env</code>	Verified
Xero	Real (Custom Connection / client_credentials)	Yes	Real draft invoice <code>INV-0377</code> pushed to Sam's Clinical Marketing Solutions Ltd tenant
Endole (credit checks)	Real v1.1 API + sandbox	Yes (sandbox)	Tesco lookup returned creditScore 38 + £1,452 CCJ
Resend (transactional email)	Real	Yes	Real email delivered (id <code>1f8e0434-55bc-4ee9-b33e-7724ca9dbbaf</code>)
SignNow (e-signature)	Real (replaced DocuSign)	Yes (trial)	Live envelope sent (id <code>18f793bf2cda4b03ad96436bbe5ac4de4b58eef7</code>)
Catchr (ad-spend MCP)	Real (MCP / JSON-RPC)	Yes	Backend starts clean; rejects when token missing

2.2 Awaiting Sam

Integration	Code	Blocker
LeadByte (lead generation)	Real, complete	Awaiting <code>LEADBYTE_API_KEY</code> from Sam — currently runs in mock mode
Cloudflare R2 (file storage)	Real (AWS SDK v3)	Sam needs to add card to Cloudflare to activate R2 billing
Resend domain	Verified <code>onboarding@resend.dev</code>	Sam's other developer needs to complete SPF/DKIM verification of <code>leadgeneration.io</code>

2.3 Available But Optional

Creditsafe is implemented as a fallback to Endole. If both are unconfigured, the credit-check service silently returns mock data. Production setup should ensure at least one is live.

3. Feature Build Status

This sprint moved the project from "service layer mostly mock" to "service layer mostly real". The big change was replacing six in-memory `MOCK_*` arrays with real Drizzle queries against new Postgres tables.

3.1 Real, Database-Backed Features

Feature	Tables	Status
Invoices	<code>invoices</code>	List, get, create, push to Xero (idempotent)
Clients	<code>clients</code> , <code>credit_checks</code>	Full CRUD; revenue summed from real paid invoices
Campaigns	<code>campaigns</code> , <code>lead_deliveries</code>	LeadByte sync; campaign-type metadata in DB
Agreements	<code>agreements</code>	SignNow envelope flow with HMAC webhook verification
Settings → Integrations	(status route)	Live "Connected" / "Not configured" per integration
Tasks	<code>tasks</code> , <code>task_comments</code> , <code>task_templates</code>	DB-backed list / get / create / update / status / comments / templates
SOPs	<code>sops</code>	DB-backed list / get / create / update with category + status filters
Staff & HR	<code>staff</code> , <code>job_postings</code> , <code>applicants</code> , <code>holiday_requests</code>	Full HR module on DB
Workflows	<code>workflows</code> , <code>workflow_executions</code>	DB-backed; BullMQ worker dispatches real handlers
Traffic Sources	<code>traffic_sources</code>	Per-campaign source aggregation
Client Portal	(joins of clients, invoices, leads, agreements)	Scoped to <code>requester.clientId</code> — security fix
Auth	<code>users</code>	JWT + refresh tokens, role-based access
Cron jobs	(BullMQ schedulers)	Chase-overdue (daily 9am), LeadByte sync (every 2min), Catchr sync (hourly)

3.2 Workflow Handler Registry

Workflows can now be bound to a named handler via the `handler_key` column. The worker dispatches to a registered handler when set, and falls back to generic step-by-step execution otherwise. Three handlers are registered today:

- `chase-overdue` — fully wired, calls the same logic as the daily cron job
- `auto-invoice` — scaffolded; identifies eligible `weekly_auto` clients but Xero push pending `lead_deliveries` population

- `monthly-validated` — scaffolded; emails Sam a per-client validation summary

User-created workflows continue to work via the generic step loop (no breaking change).

3.3 Frontend Highlights

The frontend is API-driven via React Query. Two material updates this sprint:

- The dashboard's Credit Alerts widget no longer renders a hardcoded mock list — it now calls `/api/v1/clients/credit-alerts` and renders skeleton + empty states correctly.
- The Staff "Request Holiday" form no longer pre-fills `staffId: 's-1'` (which 404'd against real UUIDs); it requires the user to pick a staff member explicitly.

4. Quality and Testing

4.1 Test Suite

The full backend test suite now passes end-to-end against a real PostgreSQL + Redis stack. Tests are self-seeding via the public API in their `beforeAll` hooks; no test relies on hand-crafted mock IDs.

Metric	Before sprint	After sprint
Tests passing	361	383
Tests failing	2	0
Tests skipped	14	0
Test files passing	11	25

The two previously-failing catchr-client tests were caused by an env-var leak (`isCatchrConfigured` reading from a module-load snapshot rather than `process.env`). Fixed in commit `a773000`.

4.2 TypeScript Health

Metric	Before sprint	After sprint
<code>npm run typecheck</code> errors	51	0

The 51 errors broke down as: vitest globals (15), Express 5 `req.params` widening (29), missing `@aws-sdk` deps (7). Fixed via tsconfig `types` config, `req.params.id as string` casts in 9 controllers, and `pnpm`

`install` to pick up declared-but-uninstalled deps.

4.3 Test Infrastructure

Test stack uses non-default ports to avoid colliding with a local production-style Postgres on 5432:

- PostgreSQL — `localhost:5433`
- Redis — `localhost:6380`
- MinIO — `localhost:9000` / `:9001` (S3-compatible storage for upload tests)

Tests run sequentially (`fileParallelism: false`) because suites share a single Postgres + Redis stack and parallel runs caused state leakage on shared business / client UUIDs.

5. Database

5.1 Schema

The database now contains 26 tables. Eight are new this sprint:

Table	Purpose
<code>sops</code>	Standard operating procedures with category + status
<code>tasks</code>	Assigned work with priority, status, due date, audit log jsonb
<code>task_comments</code>	Threaded comments per task
<code>task_templates</code>	Reusable task templates with steps array
<code>staff</code>	Internal employee records
<code>job_postings</code>	Open positions
<code>applicants</code>	Candidates per job posting
<code>holiday_requests</code>	Leave requests with approver

5.2 Migrations

Migration	Purpose	Status
0000_breezy_nick_fury	Initial schema	Applied
0001_good_grim_reaper	Schema refinements	Applied
0002_ad_spend	Catchr ad-spend table	Applied
0003_signnow_rename	Rename docusign_envelope_id → provider_envelope_id	Applied
0004_in_memory_to_db	Add 8 tables for SOPs/Tasks/Staff/HR; extend workflows + traffic_sources	Applied to local dev DB
0005_workflow_handler_key	Add handler_key column to workflows	Applied to local dev DB

All migrations are idempotent (`CREATE INDEX IF NOT EXISTS` , `ADD COLUMN IF NOT EXISTS`). Safe to re-run.

5.3 Seed Data

The seed script (`npm run db:seed`) runs in two modes:

- **Production** (`NODE_ENV=production`): seeds only the `leadgeneration.io` business + four internal users (owner / finance_admin / ops_manager / readonly). Internal user passwords come from `SEED_*_PASSWORD` env vars when set.
- **Dev / Test** (default): also seeds the demo client `Apex Media Ltd` + a `client@stato.app` portal user, plus three sample notifications.

Both demo entities use stable UUIDs (`00000000-0000-0000-0000-000000000001` for the demo client; `26d6b2b4-c867-460e-8473-eca2b1ffd232` for the business) so they line up with the in-memory user store and don't crash Postgres uuid casts.

6. Security and Compliance

6.1 Critical Fix This Sprint

The Client Portal's `getDashboard` , `getCampaigns` , `getLeads` , `getInvoices` , `getCompliance` , and `getAgreement` endpoints all returned data scoped to a hardcoded `CLIENT_ID = 'c-4'` (Delta Solutions). Every authenticated portal user — regardless of identity — was seeing the same client's data.

This was fixed in commit `2e1693d`. Every portal query is now scoped to `requester.clientId` from the JWT. A new `PortalAccessError` is thrown when the JWT is missing a `clientId`, and the controller catches it and returns 403.

6.2 Production Secret Hardening

`src/config/env.ts` now throws on startup if `NODE_ENV=production` and either `JWT_SECRET` or `JWT_REFRESH_SECRET` still equals its dev default. This makes it impossible to ship a release that signs tokens with a known-public key.

6.3 Pre-flight Check

A new `npm run preflight` script enumerates the 14 environment variables required for production (`DATABASE_URL`, `REDIS_URL`, `JWT_SECRET`, etc.) and exits non-zero if any are missing. Recommended as the last gate before promoting a Railway release.

7. Pre-Launch Operations

This sprint produced a complete production deploy procedure documented in `docs/DEPLOY-RUNBOOK.md`. Seven sections cover secrets, env vars, migrations, seeding, deployment, smoke testing, and rollback.

7.1 Required Environment Variables

Fourteen variables are required for production. Three categories:

- **Core:** `DATABASE_URL`, `REDIS_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`
- **Integration credentials:** `XERO_CLIENT_ID`, `XERO_CLIENT_SECRET`, `LEADBYTE_API_KEY`, `SIGNNOW_CLIENT_ID`, `SIGNNOW_CLIENT_SECRET`, `SIGNNOW_USERNAME`, `SIGNNOW_PASSWORD`, `CATCHR_ACCESS_TOKEN`
- **Email:** `RESEND_API_KEY`, `RESEND_FROM_EMAIL`

All have updated entries in `.env.example` with per-integration gotchas (Redis-required-for-cron, Resend domain caveat, SignNow base URL is now production by default, etc.).

7.2 Updated Default URLs

Two production-critical default values were corrected in `env.ts`:

- `SIGNNOW_BASE_URL` flipped from `api-eval.signnow.com` (sandbox) to `api.signnow.com` (production). Sandbox is now opt-in via explicit override.
-

8. What Remains

8.1 On Yash (Manual Operations Only)

1. Open pull requests from `Development` → `main` on both repositories.
2. Generate production JWT secrets (`openssl rand -base64 32` , twice).
3. Provision Railway environment (Postgres plugin, Redis plugin, env vars).
4. Run `npm run preflight` against the Railway environment.
5. Deploy backend and frontend.
6. Smoke-test `/healthz` and `/api/v1/integrations/status` post-deploy.
7. Walk through `client@stato.app` portal as a real user to confirm the security fix works end-to-end.

8.2 On Sam

1. Deliver `LEADBYTE_API_KEY` — unblocks all campaign / lead / supplier features.
2. Add a card to Cloudflare and activate R2 — unblocks file uploads and signed PDF storage.
3. Verify `leadgeneration.io` domain in Resend — unblocks branded email "from" address.
4. Upgrade SignNow workspace to a paid plan — required before 29 April 2026 trial expiry.
5. Confirm leadreports.io vs LeadByte direct integration choice (LeadByte is built; leadreports.io would require a rewrite).

8.3 Code Follow-On (Lower Priority)

- BullMQ workflow handlers for `auto-invoice` and `monthly-validated` are scaffolded but stub out the real Xero push — should be wired once `lead_deliveries` is populated by live LeadByte sync.
- The `finance/subscriptions.tsx` frontend page is still a hardcoded subscriptions list; if Sam wants this to be a real feature, it needs a `subscriptions` backend table.

9. Commit Log This Sprint

Eleven commits across two repositories, all on `Development` branch.

9.1 Backend (Sato-Backend)

Commit	Subject
42e465f	feat(integrations): swap DocuSign for SignNow + Xero Custom Connection + Endole real API
d377740	feat(db): replace mock arrays with real tables for SOPs/tasks/staff/workflows/traffic-sources
2e1693d	fix(security): scope Portal queries to requester.clientId; align demo UUIDs
2dbc95b	feat(workflows): wire BullMQ + handler registry
a773000	fix(catchr): read process.env directly so test env-clears actually work
c6652c8	chore(prod): env hardening, preflight script, deploy runbook
f31a1e1	refactor(types): cast req.params.id, add vitest globals + missing deps
87e2040	test(infra): test postgres on 5433, redis on 6380, sequential file run, self-seeding fixtures
31afe45	chore: bundle remaining prior-sprint changes (auth/permissions/notifications/tests)

9.2 Frontend (Sato-Frontend)

Commit	Subject
3330952	feat(frontend): wire credit alerts widget to API + clear staff form defaults
4180d36	chore: bundle prior-sprint frontend changes (pages, hooks, providers)

10. Recommended Next Actions

The single highest-impact action is to deploy to Railway and walk Sam through a working demo. This converts the project from "Yash thinks it works" to "Sam saw it work in production" — and that conversation is what unblocks Sam's three remaining deliveries (LeadByte key, R2 card, SignNow upgrade).

Suggested order for the next 48 hours:

1. **Today:** Open PRs on both repos. Provision Railway. Generate secrets. Deploy.
2. **Tomorrow:** 30-minute live demo with Sam covering invoices, clients, portal. Use the meeting to push his three blockers.
3. **Within 6 days:** SignNow paid plan must be activated before 29 April 2026.

After Sam's blockers are cleared, the only remaining substantive work is wiring the BullMQ `auto-invoice` and `monthly-validated` handlers to real integrations — at which point Stato is fully operational for Phase 1.